

When things touch each other.

Collisions — rect, circle, mask, group-vs-group.

Yesterday: many objects, one Group.

Day 9 turned the loose bouncing ball into a Sprite subclass and dropped N of them into a Group.

`group.update()` ran every ball's `update()`; `group.draw()` blitted them all in one call.

BEFORE — Day 9

```
# 50 balls, no awareness of each other
for ball in balls:
    ball.update()
    ball.draw(screen)

# they pass through each other
# they pass through the walls
# they pass through the paddle
```

AFTER — Day 10

```
# sprites that notice each other
balls.update()
hits = pygame.sprite.spritecollide(
    paddle, balls, False)
for b in hits:
    b.bounce_off(paddle)

# brick-breaker becomes possible
```

A collision is a yes/no question.

This page: collision detection is just "do these two shapes overlap right now?" The library doesn't bounce anything for you — it returns True/False and you decide what to do.

```
# every frame, ask the question
if ball.rect.colliderect(paddle.rect):
    # True — they overlap right now
    ball.velocity_y = -ball.velocity_y
else:
    # False — nothing to do
    pass

# detection is separate from response.
# pygame tells you 'they touch'.
# YOU write what happens next:
# bounce, vanish, damage, score, sound...
```

The two halves

DETECTION — pygame's job.
Given two things, do they overlap?
Returns a bool or a list of hits.

RESPONSE — your job.
Flip a velocity, delete a brick,
play a sound, lose a life.

colliderect — the workhorse.

This page: `pygame.Rect.colliderect` checks if two axis-aligned rectangles overlap. Fast, cheap, and accurate enough for 90% of arcade games. Every `Sprite` has a `.rect`, so this is the call you'll make constantly.

```
import pygame

ball = pygame.Rect(100, 100, 20, 20)
paddle = pygame.Rect(80, 400, 120, 15)

if ball.colliderect(paddle):
    print('HIT')
```

*# colliderect returns True the instant
any edge of one rect crosses the other.
x, y is top-left. width, height extend
down-right. that's pygame's convention.*

Why "axis-aligned"?

The rect's sides are parallel to the screen axes. Rotation isn't tracked. A rotated sprite still uses an upright bounding box.

Cost

Four comparisons. Nothing else. You can run thousands per frame without breaking 60 FPS.

When a rect lies about the shape.

This page: a ball is a circle, not a square. Its rect is the bounding box around the circle, so colliderect can fire when the corners overlap but the actual circles don't. For round things, use distance between centers vs sum of radii.

```
import math

def circles_collide(a, b):
    dx = a.x - b.x
    dy = a.y - b.y
    dist = math.sqrt(dx*dx + dy*dy)
    return dist < a.radius + b.radius

# even cheaper - skip the sqrt:
def circles_collide_fast(a, b):
    r = a.radius + b.radius
    return (a.x-b.x)**2 + (a.y-b.y)**2 < r*r
```

The trick

sqrt is slow. Compare squared distance to squared sum-of-radii. Same answer, no sqrt.

When to bother

Use circles for balls, bullets, particles. Use rects for paddles, bricks, platforms.

spritecollide — one against many.

This page: `pygame.sprite.spritecollide(sprite, group, dokill)` checks one sprite against every sprite in a group and returns the list of hits. `dokill=True` removes hits from the group — exactly what you want for bricks vanishing on impact.

```
# ball vs wall of bricks
hit_bricks = pygame.sprite.spritecollide(
    ball,          # the one sprite
    bricks,        # the group
    True,          # dokill - remove on hit
)

if hit_bricks:
    score += 10 * len(hit_bricks)
    ball.velocity_y = -ball.velocity_y
    sound_break.play()
```

Returns a list

Empty list = no hits (falsy). List with N items = N bricks were hit this frame.

dokill flag

True deletes hit sprites from the group automatically. False leaves them — use when you only want to react, not destroy.

Two more tools you'll need.

This page: groupcollide does N×M collision between two groups (bullets vs enemies). Masks are pixel-perfect — slower, but they only count actual non-transparent pixels, not the bounding box. Use masks when sprite shapes matter (e.g. an angled paddle).

groupcollide

```
# bullets vs enemies
hits = pygame.sprite.groupcollide(
    bullets, enemies,
    True, True # kill both
)

# returns dict:
# {bullet1: [enemy1], ...}
for bullet, killed in hits.items():
    score += 100 * len(killed)
```

Mask (pixel-perfect)

```
# in __init__:
self.image = pygame.image.load(...)
self.mask = pygame.mask.from_surface(
    self.image)

# in collision check:
if pygame.sprite.collide_mask(
    ball, paddle):
    # real pixels overlap
    ball.bounce()
```

Pick the cheapest check that works.

Method	Best for	Cost	Catch
<code>colliderect</code>	rect vs rect	4 comparisons	<i>boxy, ignores shape</i>
<code>collide_circle</code>	balls, bullets	distance + sqrt	<i>needs a .radius attr</i>
<code>spritecollide</code>	1 sprite vs group	N rect checks	<i>default is rect-based</i>
<code>groupcollide</code>	group vs group	N × M rect checks	<i>returns dict</i>
<code>collide_mask</code>	irregular shapes	per-pixel, slow	<i>mask must stay synced</i>

Rule of thumb: always start with colliderect. Upgrade only when you can see false hits.

Brick-breaker prototype.

Mouse-controlled paddle, one bouncing ball, a wall of bricks. Hit a brick → it disappears, ball reverses. Miss the ball → game over. No sounds, no scenes — that's Days 11 and 12.

STRUCTURE

- `Paddle(pygame.sprite.Sprite)`
- `Ball(pygame.sprite.Sprite)`
- `Brick(pygame.sprite.Sprite)`
- `bricks = pygame.sprite.Group()`
- build an 8×5 grid of bricks

COLLISIONS

- `ball↔paddle` → `colliderect`, flip `vy`
- `ball↔bricks` → `spritecollide(True)`
- `ball↔walls` → flip `vx` at edges
- `ball↔floor` → game over
- `paddle.x = mouse.x` (clamp it)

DELIVERABLE

- `brick_breaker.py`
- 640 × 480, 60 FPS
- all bricks gone → win text
- ball off bottom → lose text
- ESC quits, SPACE resets

What goes wrong first.

⚠ Move first, then check — not the other way round.

If you detect first and then translate, you can land inside the brick. Update positions, then collide, then respond by reversing velocity — not by moving the ball back.

⚠ Multiple bricks in one frame is normal.

`spritecollide` returns a list. If two bricks both touch the ball corner, you'll get both. Flip velocity once, not once per brick — or the bounce cancels.

💡 A Sprite needs `self.image` and `self.rect` — that's it.

pygame's collision functions read `.rect` off every sprite. If you forget to set it in `__init__`, `spritecollide` silently returns nothing. No error, no hits.

💡 Clamp the paddle inside the window.

Mouse can leave the screen edges. `paddle.rect.x = max(0, min(W - paddle.rect.w, mouse_x))` keeps it visible and keeps the ball bounceable.

W R A P - U P · W H A T T H I S F E E D S

Today you got a game to react. Next, it needs feel.

DAY 11 · THU

Sound + animation

pygame.mixer for hit/break/lose. Frame-based paddle squish. Brick-breaker stops feeling sterile.

DAY 12 · FRI

Scene state machine

Menu → play → game-over. Scene base class wraps the loop. Brick-breaker gets a launcher.

DAY 13 · SAT

Pong (proper)

Two paddles, angle-based deflection, AI right paddle, scoring, menu wired in. Week 2 keystone.

Detection is cheap. Response is where the game lives.